2b Academy — Python + Full Stack Basics + AI Fundamentals

Detailed Day-Wise, Hour-Wise Training Plan

Batch Dates: 1 July – 31 July 2026

Total Duration: 22 training days, 5 days/week (Mon–Fri) | **3 hours/day** | **66 contact hours** *(Assumption: Mon–Fri working week. If your institute runs Tue–Sat or includes Saturdays, tell me and I'll re-map the dates — the day-by-day content underneath doesn't change, only the calendar labels do.)*

---

0. Calendar Note

July 2026 has **23 weekdays** (Mon–Fri) between the 1st and 31st. Your curriculum's 7 modules already total **exactly 22 days** (6+2+3+3+4+2+2), so no days were added or removed this time — I just laid them onto the real calendar starting Wednesday, 1 July.

That leaves **one weekday unused: Friday, 31 July.** I left it as a deliberate **buffer/flex day** rather than guessing which date your institute treats as a holiday — use it for a public holiday, a make-up class for anyone absent during the month, extra project time, or simply close the batch a day early. Swap it to any other date in the month if you'd rather the buffer fall mid-month.

Two smaller adjustments carried over from the structure, both noted again here for clarity:

- **Canva AI** is a 10-minute demo on Day 19, not a taught skill — it's a design tool, not core to the dev/AI outcomes.
- **Tuples** are folded into the Lists lesson (Day 4) rather than given separate weight.
- There's no separate "review day" this time (to keep the day count at 22) — instead, a 5–10 minute recap quiz opens Day 1 of each new module, and the Module 7 days do double duty as career guidance *and* final project *and* demo day (see Days 21–22).

---

1. Calendar-at-a-Glance

Day #	Date	Weekday	Module	Topic
1	Jul 1	Wed	Module 1	Intro to Python + Python Basics (Setup, variables, I/O, operators)
2	Jul 2	Thu	Module 1	Control Statements (if-else, loops) + Number Guessing Game

Day #	Date	Weekday	Module	Topic
3	Jul 3	Fri	Module 1	Functions
—	Jul 4–5	Sat/Sun	—	Weekend
4	Jul 6	Mon	Module 1	Data Structures (Lists, Tuples, Dictionaries) + Student Marks Program
5	Jul 7	Tue	Module 1	String Handling + Pattern Programs
6	Jul 8	Wed	Module 1	File Handling + Error Handling + Module 1 Capstone
7	Jul 9	Thu	Module 2	OOP Part 1: Classes, Objects, Constructors
8	Jul 10	Fri	Module 2	OOP Part 2: Inheritance, Encapsulation + Project
—	Jul 11–12	Sat/Sun	—	Weekend
9	Jul 13	Mon	Module 3	HTML Basics + Portfolio Page (start)
10	Jul 14	Tue	Module 3	CSS Basics + Responsive Form Design
11	Jul 15	Wed	Module 3	JavaScript Basics + Interactive Buttons + Finalize Portfolio
12	Jul 16	Thu	Module 4	Backend Concepts + Flask Basics
13	Jul 17	Fri	Module 4	Templates, Form Handling + Login Form
—	Jul 18–19	Sat/Sun	—	Weekend
14	Jul 20	Mon	Module 4	Dynamic Data + Contact Form + Flask Website Capstone
15	Jul 21	Tue	Module 5	Intro to AI + NumPy Basics

Day #	Date	Weekday	Module	Topic
16	Jul 22	Wed	Module 5	Pandas Basics + Data Handling
17	Jul 23	Thu	Module 5	Matplotlib + Data Visualization
18	Jul 24	Fri	Module 5	AI & ML Basics + Student Result Prediction Capstone
—	Jul 25–26	Sat/Sun	—	Weekend
19	Jul 27	Mon	Module 6	Generative AI Intro + Prompt Engineering + AI Tools Overview
20	Jul 28	Tue	Module 6	AI Ethics + Practical GenAI Use (Content, Chatbot, Resume)
21	Jul 29	Wed	Module 7	Git/GitHub + Deployment + Career Guidance (Part 1)
22	Jul 30	Thu	Module 7	Career Guidance (Part 2) + Final Project + Demo Day
—	Jul 31	Fri	**Buffer**	Flex day — holiday / make-up class / extra demo time

---

2. The 3-Hour Daily Rhythm (Use This Every Single Day)

This is the engagement engine of the whole bootcamp. Don't treat the 3 hours as "lecture, lecture, lecture" — attention collapses after ~25 minutes of pure talking, especially for a beginner audience sitting for 3 hours straight.

Hour 1 — "Learn It" (Concept + Live Demo)

Time	What happens
0–8 min	**Hook.** Recap yesterday in 2 minutes via rapid-fire Q&A ("one word — what's a variable?"), then a 1-line real-world story for today's topic ("Netflix's recommendation engine literally

Time	What happens
	starts with the loop you're about to learn"). On Day 1 of a new module, this slot becomes a 5–10 min recap quiz of the previous module instead.
8–35 min	**New concept**, taught with an analogy first, then on-screen live coding — you type, they watch, nobody codes yet. Narrate your thinking out loud ("I'm getting a TypeError here — why?") so debugging feels normal, not scary.
35–55 min	**Code-along.** Same example, now they type it with you, line by line, on their own machines. You watch screens/walk the room.
55–60 min	**60-second doubt round** — only "I'm stuck," no broader discussion (save that for Hour 2).

Hour 2 — "Do It" (Hands-On Lab)

Time	What happens
0–10 min	Assign 2–3 short independent problems (not the demo example — force them to apply it differently).
10–45 min	**Pair programming**, roles swap every ~15 min (driver/navigator). You float and debug 1-on-1. Struggling pairs get pulled aside; fast finishers get a stretch bonus problem.
45–55 min	**2 students screen-share their solution** (rotate who presents each day so everyone gets a turn over the month).
55–60 min	You name the **top 2 mistakes** you saw across the room — normalizes errors, teaches the whole class at once.

Hour 3 — "Use It" (Apply + Close)

Time	What happens
0–35 min	Work on the **day's Practical/Mini-Project** (from your curriculum's "Practical Concepts" lists) — this is where the day's concept gets used in something real, not a toy exercise.
35–45 min	**Peer code review** — swap laptops with a neighbor, leave one comment ("what would you improve?").
45–55 min	Recap on whiteboard/slide: 3 bullet takeaways. Preview tomorrow with a 1-line

Time	What happens
	teaser/cliffhanger.
55–60 min	**Exit ticket** — each student writes one sentence: "today I learned ___" or one remaining doubt. You read these before the next class to plan re-teaching.

Engagement Toolkit (rotate through these so it doesn't feel repetitive)

- **Daily rapid-fire quiz** (verbal or Kahoot-style) at the start — costs nothing, wakes the room up.
- **Pair programming with role swaps** — keeps both students active instead of one typing/one watching.
- **"Bug hunt" challenges** — give them broken code, first to fix it wins a shout-out.
- **Mini leaderboard** for lab exercises (track on a whiteboard — gamifies without being heavy-handed).
- **Rotating presenter** — every student must screen-share their work at least 2–3 times across the month; removes the "same 3 confident students always talk" problem.
- **Real-world tie-in** at the start of each new topic ("this is literally how Instagram's login works").
- **5-minute stretch/break** between Hour 1→2 and Hour 2→3 — non-negotiable for a 3-hour block; attention craters without it.
- **End-of-week wrap (last 15–20 min of each Friday)**: review the *week*, not just the day — this is what actually moves things into long-term memory.

WEEK 1 (Jul 1 – Jul 3) — Python Fundamentals Begin

Day 1 — Wed, Jul 1: Orientation + Intro to Python + Python Basics (Part 1)

Goal: Every student leaves having installed their environment and run their first programs.

Hour 1 — Setup & What is Python

- Icebreaker: name + one personal goal for the month.
- What is Python: interpreted language, why it's beginner-friendly (readability, huge ecosystem).
- Applications of Python: web dev, automation, data science, AI/ML, game dev — one real example per category.
- Installing Python + VS Code live, on the projector — verify with `python --version`.

- Installing the Python extension in VS Code; running via Run button vs terminal vs interactive shell.
- **Activity:** everyone writes and runs `print("Hello, my name is ___")`.

Hour 2 — Variables, Data Types, I/O

- Variables: naming rules, dynamic typing, case sensitivity.
- Core data types: `int`, `float`, `str`, `bool`, `None`; `type()`.
- Input/Output: `input()`, `print()`, f-strings.
- **Lab:** "Mad Libs" style program using f-strings.

Hour 3 — Operators, Type Conversion + Mini Project Kickoff

- Arithmetic, comparison, logical, assignment operators; quick note on precedence.
- Type conversion: `int()`, `float()`, `str()`.
- **Practical:** start the **Calculator Program** — two numbers + an operator as input, compute, print result.
- Exit ticket + preview tomorrow.

Day 2 — Thu, Jul 2: Control Statements

Goal: Students can write decision-making and repeating logic confidently.

Hour 1 — If-Else & Nested Conditions

- `if`, `elif`, `else` syntax and flow.
- Nested conditions vs `elif` chains (readability trade-off).
- Live demo: extend yesterday's calculator to handle invalid operators.

Hour 2 — Loops (for, while)

- `for` loops over ranges/strings; `while` loops and infinite-loop risk; `break`/`continue`.
- **Lab:** multiplication tables, sum of digits, counting problems.

Hour 3 — Practical: Number Guessing Game

- Build the **Number Guessing Game**: `random` module, `while` loop until correct guess, hint logic.
- Leaderboard for cleanest logic. Recap + exit ticket.

Day 3 — Fri, Jul 3: Functions

Goal: Students can write and call their own reusable functions correctly.

Hour 1 — Creating Functions & Parameters

- Why functions exist (DRY principle).
- `def`, parameters, default values, return values vs `print` inside a function.
- Live demo: refactor the calculator into a function.

Hour 2 — Built-in Functions + Lab

- `len()`, `range()`, `sum()`, `max()`, `min()`, `sorted()`.
- **Lab:** write `is_even()`, `factorial()`, `reverse_string()` — pair programming with role swap.

Hour 3 — Applied Practice

- Bug hunt: broken function code on screen.
- Mini-challenge: function-based **Quiz Scorer**.
- **End-of-week recap (last 15 min):** Week 1 so far — variables → control flow → functions.

WEEK 2 (Jul 6 – Jul 10) — Data Structures, Strings, Files, OOP

Day 4 — Mon, Jul 6: Data Structures

Goal: Comfortable choosing between lists, tuples, and dictionaries for a given problem.

Hour 1 — Lists & Tuples

- Lists: creation, indexing, slicing, methods (`append`, `remove`, `sort`).
- Tuples: immutability, when to use one over a list (10 min, not a full topic).
- Live demo: store/process a list of student names.

Hour 2 — Dictionaries + Lab

- Key-value pairs, `.keys()`/`.values()`/`.items()`.
- **Lab:** contact book using a dictionary — add/search/delete.

Hour 3 — Practical: Student Marks Program

- Store names+marks in a dictionary, compute average/highest/lowest, print a report.
- Pair review + 2 students present.

Day 5 — Tue, Jul 7: String Handling

Goal: Manipulate and pattern-match strings comfortably.

Hour 1 — String Operations & Functions

- Indexing/slicing, concatenation, immutability.
- `.upper()`, `.lower()`, `.strip()`, `.split()`, `.replace()`, `.find()`.

- Live demo: clean messy user input.

Hour 2 — Lab: String Manipulation

- Palindrome checker, word counter, vowel counter.

Hour 3 — Practical: Pattern Programs

- Triangle/number-pyramid patterns using nested loops + string repetition.
- Leaderboard for cleanest pattern code.

Day 6 — Wed, Jul 8: File Handling, Error Handling + Module 1 Capstone

Goal: Wrap up Module 1 with a project combining the whole module.

Hour 1 — File Handling Basics

- `open()`, modes (`r`/`w`/`a`), reading line by line, `with open(...)`.
- CSV basics with the `csv` module.

Hour 2 — Error Handling

- `try/except`, specific exceptions (`ValueError`, `ZeroDivisionError`).
- "Greatest hits" of common errors from the week.
- **Lab:** error-proof Day 1's calculator.

Hour 3 — Practical: File-Based Mini Project (Module 1 Capstone)

- Student records system: read names/marks from CSV, compute stats, write a results report, with error handling.
- Quick check-in on every student's working version.

Day 7 — Thu, Jul 9: OOP Part 1 — Classes, Objects, Constructors

Goal: Understand why OOP exists and write basic classes.

Hour 1 — Classes & Objects

- Why OOP — bundling data + behavior (contrast with Day 6's dictionary-based approach).
- `class` syntax, instances, instance attributes.
- Live demo: a `Student` class with name and marks.

Hour 2 — Constructors + Lab

- `__init__()`, `self` keyword (most common confusion point).
- **Lab:** `Book`, `Car`, `Employee` classes with constructors + one method each.

Hour 3 — Applied Practice

- Extend `Student` with `get_average()`, `is_passing()`.
- Bug hunt: a class missing `self`.

Day 8 — Fri, Jul 10: OOP Part 2 — Inheritance, Encapsulation + Project

Goal: Apply OOP to a complete small program.

Hour 1 — Inheritance

- Parent/child classes, `super()`, method overriding.
- Live demo: `Person` → `Student`/`Teacher`.

Hour 2 — Encapsulation + Lab

- Public vs "private" attributes (_/__ convention).
- **Lab:** encapsulated balance handling on an `Account` skeleton.

Hour 3 — Practical Project (pick ONE as the graded build)

- **Option A — Student Management Program**: add/update/search records, computed grades.
- **Option B — Simple Banking System**: deposit/withdraw/balance, encapsulated balance, validation.
- Whichever isn't chosen is offered as a bonus for fast finishers.
- **End-of-week recap:** Week 2 — data structures → strings → files → OOP.

WEEK 3 (Jul 13 – Jul 17) — Frontend Basics, Backend Begins

Day 9 — Mon, Jul 13: HTML Basics

Goal: Build a structured static HTML page.

Hour 1 — HTML Structure & Text

- `<html>`, `<head>`, `<body>`; headings, paragraphs, text formatting.
- Live demo: scaffold a personal bio page.

Hour 2 — Links, Images, Tables, Forms

- `<a>`, `<img>` (with alt text), `<table>`, `<form>` basics (no submission logic yet).
- **Lab:** add project links + an image to the bio page.

Hour 3 — Practical: Personal Portfolio Page (Start)

- Header/about/projects/contact sections in plain HTML — styling comes tomorrow.

Day 10 — Tue, Jul 14: CSS Basics

Goal: Style the portfolio page and understand layout fundamentals.

Hour 1 — Colors, Fonts, Styling Basics

- Inline vs internal vs external CSS; selectors; color, typography.
- Live demo: style the bio page headers/text.

Hour 2 — Box Model + Lab

- Margin, border, padding, content.
- **Lab:** card-style layout for the "projects" section.

Hour 3 — Practical: Responsive Basics + Form Styling

- Viewport meta tag, simple media queries.
- **Practical: Responsive Form Design** — style the contact form to work on mobile.

Day 11 — Wed, Jul 15: JavaScript Basics

Goal: Add interactivity to the portfolio page.

Hour 1 — Variables, Functions, Events

- `let`/`const`, basic functions, arrow function intro.
- Event listeners (`onclick`/`addEventListener`).
- Live demo: a button that changes text on click.

Hour 2 — DOM Manipulation + Lab

- `document.querySelector`, `.innerText`/`.innerHTML`, changing styles via JS.
- **Lab:** dark-mode toggle button.

Hour 3 — Practical: Interactive Buttons + Finalize Portfolio

- Add interactive elements (carousel toggle, "show more" bio, form validation message).
- Finalize the **Personal Portfolio Page** — this gets deployed in Week 5.
- 2–3 students present. **End-of-week recap:** Week 3 — HTML → CSS → JS.

Day 12 — Thu, Jul 16: Backend Concepts + Flask Basics

Goal: Understand client-server architecture and get a Flask app running.

Hour 1 — Introduction to Backend

- Frontend vs backend, using the portfolio page as the example.
- Client-server basics: request/response cycle.

- APIs introduction, conceptually (restaurant-menu analogy).

Hour 2 — Installing Flask + First App

- `pip install flask`, minimal app, running the dev server.
- **Lab:** every student gets "Hello Flask" running locally.

Hour 3 — Routing + Practice

- Multiple routes, route parameters (`/user/<name>`).
- **Lab:** 3–4 routes returning different text/JSON responses.

Day 13 — Fri, Jul 17: Templates, Form Handling

Goal: Connect frontend pages to a Flask backend.

Hour 1 — Templates

- Jinja templates, `render_template`, passing variables.
- Live demo: serve the portfolio's "about" section dynamically.

Hour 2 — Taking User Input + Lab

- GET vs POST, `request.form`.
- **Lab:** a form that submits a name and Flask greets the user back.

Hour 3 — Practical: Login Form

- HTML form → Flask route → basic validation (hardcoded check) → success/failure message.
- Call out the milestone: frontend + backend connected end-to-end for the first time.
- **End-of-week recap:** Week 4 so far — client-server → Flask → forms.

WEEK 4 (Jul 20 – Jul 24) — Backend (Flask) + AI Fundamentals

Day 14 — Mon, Jul 20: Dynamic Data + Module 4 Capstone

Goal: Build a small but complete Flask website.

Hour 1 — Displaying Dynamic Data

- Passing lists/dictionaries into templates, looping inside Jinja (`{% for %}`).
- Live demo: render a dynamic "projects" list from a Python list.

Hour 2 — Practical: Contact Form Backend

- Form submits to a Flask route, data validated and displayed back or stored.

Hour 3 — Practical: Simple Flask Website (Module 4 Capstone)

- Combine everything: a small multi-page Flask site — home page, dynamic project list, working contact form.
- 2 students present.

Day 15 — Tue, Jul 21: Intro to AI + NumPy Basics

Goal: Build conceptual AI literacy and start the Python data-science toolkit.

Hour 1 — Introduction to AI

- What is AI — AI vs ML vs Deep Learning, plain-language.
- Real-world AI applications: recommendations, voice assistants, fraud detection, image recognition.
- Machine learning basics: learning patterns from data (spam email example).

Hour 2 — NumPy Basics

- Arrays vs Python lists (speed, vectorized ops).
- Creating arrays, indexing/slicing, `.sum()`, `.mean()`.
- **Lab:** basic array stats exercises.

Hour 3 — Applied Practice

- Recompute Week 2's Student Marks averages using NumPy instead of manual loops.

Day 16 — Wed, Jul 22: Pandas Basics + Data Handling

Goal: Load and clean real tabular data.

Hour 1 — Pandas Basics

- DataFrames vs Series, `.head()`, `.info()`, `.describe()`.
- Live demo: load and explore a sample CSV dataset.

Hour 2 — Reading CSV Files + Data Cleaning

- `pd.read_csv()`, missing values (`.isnull()`, `.dropna()`, `.fillna()`), fixing data types.
- **Lab:** clean a deliberately messy sample dataset.

Hour 3 — Applied Practice

- Filtering/grouping basics (`.groupby()`, simple conditions).

Day 17 — Thu, Jul 23: Matplotlib + Data Visualization

Goal: Turn cleaned data into visual insight.

Hour 1 — Matplotlib Basics

- Line, bar, scatter, histogram — when to use which.
- Live demo: plot a trend from the cleaned dataset.

Hour 2 — Data Visualization Lab

- **Lab:** 2–3 different charts, labeled properly.
- Peer review: "what story does this chart tell?"

Hour 3 — Practical: CSV Analysis + Simple Graph Visualization

- Full mini-analysis: load → clean → compute stats → visualize, on a fresh dataset.

Day 18 — Fri, Jul 24: AI & ML Basics + Module 5 Capstone

Goal: Understand classification/regression conceptually and see a model trained, even if simplified.

Hour 1 — Classification & Regression

- Classification vs regression, with examples.
- Model training concepts: training data, features/labels, "learning a pattern" — kept conceptual.

Hour 2 — Guided Demo: Simple Model

- Live demo using `scikit-learn` at a basic level (e.g., predicting pass/fail from study hours) — focus on workflow, not math.
- **Lab:** students tweak the same example with different inputs.

Hour 3 — Practical: Student Result Prediction (Module 5 Capstone)

- Simple model predicting pass/fail or score range from study hours/attendance.
- **End-of-week recap:** Week 4 — Flask backend → AI/data toolkit.

WEEK 5 (Jul 27 – Jul 31) — Generative AI, GitHub, Career, Final Project

Day 19 — Mon, Jul 27: Generative AI & AI Tools

Goal: Conceptual understanding of GenAI + hands-on exposure to common tools.

Hour 1 — Introduction to Generative AI

- What is Generative AI, how LLMs differ from traditional ML — plain-language, no heavy math.
- LLM basics: what makes tools like ChatGPT possible, conceptually.

Hour 2 — Prompt Engineering Basics

- Being specific, giving context/examples, iterating on output.
- **Lab:** write prompts for a given task, compare results in pairs.

Hour 3 — AI Tools Overview

- Hands-on tour: **ChatGPT**/**Gemini** for coding help; **GitHub Copilot** in-editor demo; **10-minute demo only** of **Canva AI** (awareness, not a taught skill).

Day 20 — Tue, Jul 28: AI Ethics + Practical GenAI Use

Goal: Use AI tools responsibly and productively for real tasks.

Hour 1 — AI Ethics Basics

- Responsible AI usage: verifying output, not blindly trusting it.
- AI limitations: hallucinations, bias.
- Data privacy basics: what not to paste into AI tools.

Hour 2 — Practical: AI Content Generation + Chatbot Demo

- Draft a project README using a chat tool, then have students edit it themselves ("AI as a draft, not a final answer").
- Quick conceptual look at a basic chatbot flow.

Hour 3 — Practical: Resume Generation Using AI

- Draft a first-pass resume with an AI tool, then move into manual refinement — sets up Day 22's career session.

Day 21 — Wed, Jul 29: Git, GitHub, Deployment + Career Guidance (Part 1)

Goal: Every student has their portfolio live on the internet, and starts on their resume.

Hour 1 — Git & GitHub Basics

- Why version control matters.
- `git init`/`add`/`commit`, connecting to GitHub.
- Live demo end-to-end: local folder → pushed GitHub repo.

Hour 2 — Push & Pull + Deployment

- `git push`/`git pull`, basic conflict awareness.
- **GitHub Pages**: deploy the static portfolio directly from the repo.
- **Intro to Netlify**: brief alternative deploy demo.
- Milestone: everyone has a live, shareable URL by the end of this hour.

Hour 3 — Career Guidance Part 1: Developer Roles + Resume Building

- Entry-level role overview (frontend/backend/full-stack/QA/support) — realistic expectations.
- Resume building: structuring projects/skills, using the Day 20 AI-assisted draft as a starting point.

Day 22 — Thu, Jul 30: Career Guidance (Part 2) + Final Project + Demo Day

Goal: Apply everything learned into one complete project, present it, and leave interview-ready.

Hour 1 — Career Guidance Part 2: Portfolio Building + Interview Prep

- Using the deployed GitHub/portfolio as an interview talking point; writing good project descriptions/READMEs.
- Common entry-level technical + HR-style questions; quick pair mock-interview round.

Hour 2 — Final Project Build

- Students choose ONE track:
    1. **Python Mini Project** (extended file-based app using OOP + error handling).
    2. **Basic Full Stack Web App** (Flask + HTML/CSS/JS).
    3. **AI-Integrated Mini Application** (calls an AI tool/API or uses the Day 18 ML workflow).
- Push to GitHub and deploy if applicable.

Hour 3 — Demo Day

- Each student presents live (~3–4 min): what it does, what they learned, one challenge overcome.
- Instructor + peer feedback after each demo.
- Closing: certificate distribution, recap of the full month's journey, next-step guidance (internships, continued learning).

Fri, Jul 31 — Buffer / Flex Day

No new content scheduled. Use for: a public holiday, make-up sessions for absentees, extra demo-day overflow, or simply close the batch early.

---

3. Tools & Setup Checklist (Confirm Before Day 1)

- Python 3.x installed on every machine
- VS Code + Python extension

- Stable internet for installs (`pip install flask pandas numpy matplotlib scikit-learn`)
- GitHub account created for every student **before Day 21** (assign as pre-work)
- Access to ChatGPT/Gemini for Days 19–20 (free tiers are sufficient)

4. Assessment & Capstone Summary

Checkpoint	**Date**	**Format**
Module 1 Capstone	Jul 8 (Day 6)	File-based student records mini project
Module 2 Capstone	Jul 10 (Day 8)	Student Management Program or Banking System
Module 4 Capstone	Jul 20 (Day 14)	Simple Flask multi-page website
Module 5 Capstone	Jul 24 (Day 18)	Student Result Prediction mini project
Final Capstone + Demo	Jul 30 (Day 22)	Student's choice of 3 project tracks, presented live